Postcondition

```
this->valid()==false
```

D.4.2. std::shared_future class template

The `std::shared_future` class template provides a means of waiting for an asynchronous result from another thread, in conjunction with the `std::promise` and `std::packaged_task` class templates and `std::async` function template, which can be used to provide that asynchronous result. Multiple `std::shared_future` instances can reference the same asynchronous result.

Instances of `std::shared_future` are `CopyConstructible` and `CopyAssignable`. You can also move-construct a `std::shared_future` from a `std::future` with the same `ResultType`.

Accesses to a given instance of `std::shared_future` aren't synchronized. It's therefore *not safe* for multiple threads to access the same `std::shared_future` instance without external synchronization. But accesses to the associated state are synchronized, so it *is* safe for multiple threads to each access separate instances of `std::shared_future` that share the same associated state without external synchronization.

Class definition

```
template<typename ResultType>
class shared_future
{
public:
    shared_future() noexcept;
    shared_future(future<ResultType>&&) noexcept;

    shared_future(shared_future&&) noexcept;
    shared_future(shared_future const&);
    shared_future& operator=(shared_future const&);
    shared_future& operator=(shared_future&&) noexcept;
    ~shared_future();
```